Univerzita Karlova
Přírodovědecká fakulta

Algoritmy počítačové kartografie

Úloha č. 1: Geometrické vyhledávání bodu

Matěj Hrabal, Kateřina Spudilová
1. n-gkdpz

Praha 2026

Zadání

Úloha č. 1: Geometrické vyhledávání bodu

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, ..., P_n\}$, *analyzovaný bod* q.

Výstup: P_i, $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q.

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

Hodnocení:

Krok	**Hodnocení**
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.	*+10b*
Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.	*+2b*
Zvýraznění všech polygonů pro oba výše uvedené singulární případy.	*+3b*
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	*+10b*
Řešení pro polygony s dírami.	*+10b*
*Načtení vstupních dat ze *.shp.*	*+10b*
Max celkem:	**65b**

Čas zpracování: 2 týdny.

Údaje o bonusových úlohách

Ze zadaných kroků byly řešeny následující:

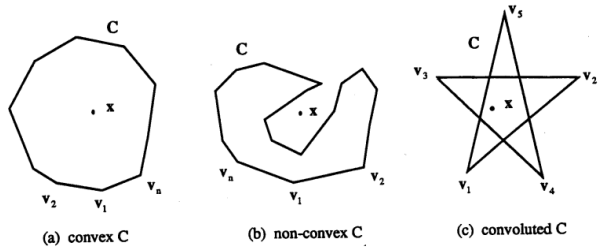
- Detekce bodu rozlišující stavy uvnitř, vně polygonu. (10b)
- Analýza polohy bodu (uvnitř/vně) metodou *Winding Number Algorithm.* (+10b)
- Ošetření singulárního případu u *Winding Number Algorithm*: bod leží na hraně polygonu. (+5b)
- Ošetření singulárního případu u *Ray Crossing Algorithm*: bod leží na hraně polygonu. (+5b)
- Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů. (+2b)
- Zvýraznění všech polygonů pro oba výše uvedené singulární případy. (+3b)

Teoretický rozbor problému

Point Location Problem

Point Location Problem, také označovaný jako *Point in Polygon* je problémem řešícím prostorový vztah mezi určitým bodem a polygonem. Je široce využíván zejména v oblasti počítačové grafiky, robotiky či geoinformačních systémů (GIS). Jeho řešení pak odpovídá na otázku, zda je daný bod q prvkem polygonu P.

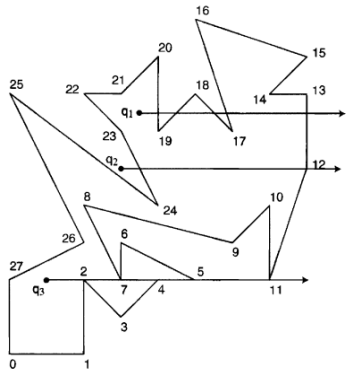
Obtížnost řešení tohoto problému podstatně závisí na konvexitě polygonu. Za konvexní polygon lze považovat takový polygon P, kde úsečka AB leží uvnitř P a dané dva body A, B jsou součástí P. Zároveň pro takový polygon platí, že žádný vnitřní úhel není větší než 180° a každá přímka, na které leží hrana polygonu, definuje polorovinu, v níž leží celý zbytek tohoto polygonu. Díky využití těchto vlastností je řešení Point Location Problem pro konvexní polygony výpočetně méně náročné než u polygonů nekonvexních. Toho využívá optimalizovaná verze tzv. *LeftOn testu*, kdy lze s využitím binárního dělení hran dosáhnout logaritmické složitosti $O(\log n)$. Pro obecné nekonvexní polygony však tato optimalizace není uplatnitelná a je nutné využít univerzální algoritmy s lineární složitostí $O(n)$, jako jsou *Ray Crossing* či *Winding Number*.



Obrázek 1: Typy polygonů z hlediska konvexity (Alciatore, Miranda 1995)

Ray Crossing algoritmus

Ray Crossing algoritmus, též známý jako *Ray Casting*, je jednou z nejpoužívanějších metod řešících *Point Location Problem*. Jeho princip je založen na platnosti Jordanovy věty o křivce. Ta říká, že každá jednoduchá uzavřená křivka v rovině rozděluje rovinu na právě dvě disjunktní oblasti: vnitřní (ohraničenou) a vnější (neohraničenou). Z této věty pak vyplývá zásadní topologická vlastnost: jakákoliv polopřímka vycházející z bodu uvnitř křivky má s hranicí polygonu lichý počet průsečíků, aby se dostala do vnější oblasti. Naopak polopřímka vycházející z vnější oblasti má s hranicí polygonu sudý počet průsečíků.



Obrázek 2: Princip *Ray Crossing* algoritmu (O'Rourke 2005)

$$
\begin{aligned}
q_1 : k_1 = 5 &\implies k_1 \equiv 1 \pmod 2 \implies q_1 \in P \\
q_2 : k_2 = 2 &\implies k_2 \equiv 0 \pmod 2 \implies q_2 \notin P \\
q_3 : k_3 = 5 &\implies k_3 \equiv 1 \pmod 2 \implies q_3 \in P
\end{aligned}
$$

Samotný algoritmus vede z analyzovaného bodu $q(x_q, y_q)$ paprsek (polopřímku) v libovolném směru. Následně je testována každá hrana polygonu P a vyhodnocen její počet průsečíků s paprskem.

$$
\begin{aligned}
q \in P &\iff k \equiv 1 \pmod 2 \\
q \notin P &\iff k \equiv 0 \pmod 2
\end{aligned}
$$

Nevýhodou základní implementace *Ray Crossing* algoritmu je jeho náchylnost k chybám v singulárních případech. To jsou případy, kdy paprsek prochází vrcholem polygonu, bod leží přímo na jeho hraně nebo polygon obsahuje vodorovné hrany. Pro zajištění robustnosti řešení je proto nutné implementovat dodatečné logické podmínky, které tyto stavy detekují a ošetřují.

Winding Number algoritmus

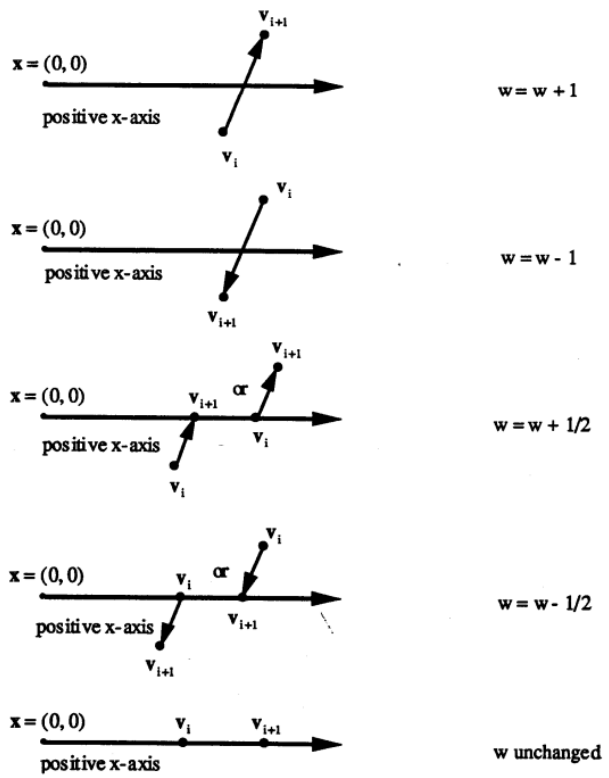
Podstata tohoto algoritmu tkví v analogii pozorovatele stojícího v testovaném bodě q, který sleduje bod p vykonávající kompletní oběh po hranici polygonu v protisměru hodinových ručiček. Zatímco bod p prochází jednotlivé segmenty, pozorovatel se plynule otáčí tak, aby jej neustále sledoval, přičemž po návratu do výchozí pozice musí být celkový úhlový posun ω celočíselným násobkem plného kruhu. Pokud se bod q nachází uvnitř polygonu, pozorovatel se otočí o 2π radiánů nebo jejich násobek, zatímco u bodů vně polygonu je výsledný součet orientovaných úhlů roven nule.

Matematicky je oběhové číslo Ω definováno jako podíl celkové sumy orientovaných rotací ω_i nad všemi vrcholy polygonu a hodnoty 2π.

$$\Omega(q, P) = \frac{1}{2\pi} \sum_{i=1}^{n} \omega(p_i, q, p_{i+1})$$

Tento vztah vyžaduje určení orientace každého úhlu, k čemuž se v praktických algoritmech využívá test polohy bodu q vzhledem k orientované přímce tvořené hranou (p_i, p_{i+1}). Tato orientace je exaktně dána znaménkem determinantu t, který určuje, zda bod q leží v levé či pravé polorovině vzhledem k dané hraně.

Vzhledem k vysoké výpočetní náročnosti goniometrických funkcí se však v praktických implementacích dává přednost diskrétnímu přístupu k výpočtu oběhového čísla. Tato metoda transformuje problém na sledování křížení hran polygonu s pevnou polopřímkou (obvykle kladnou poloosou x) v lokálním souřadnicovém systému, kde testovaný bod q představuje jeho počátek. Jednotlivým hranám jsou přiřazeny hodnoty na základě směru jejich průchodu přes tuto poloosu: hrana směřující vzhůru přičítá k celkové sumě hodnotu 1, zatímco hrana směřující dolů hodnotu 1 odčítá. Bod je následně považován za vnitřní prvek polygonu tehdy, pokud je výsledná suma (tedy hodnota Ω) nenulová.



Obrázek 3: Diskrétní přístup k výpočtu *Winding Number* (Alciatore, Miranda 1995)

Popis implementace algoritmů

Za účelem řešení této úlohy byla vyvinuta interaktivní aplikace v programovacím jazyce Python ve spojení s frameworkem *PyQt6*. Vnitřní reprezentace prostorových dat v aplikaci využívá špagetový datový model, v němž jsou polygony chápány jako nezávislé geometrické entity definované uspořádaným seznamem svých vrcholů. Aplikace umožňuje uživateli interaktivní zadávání dat, vizualizaci výsledků prostorových dotazů a přepínání mezi dvěma řešenými algoritmy - *Ray Crossing* a *Winding Number*.

Příprava dat je řešena ve třídě `Draw`, v níž je implementován seznam polygonů `self.__polygons`, do kterého je každý uživatelem nakreslený útvar uložen jako objekt `QPolygonF`. K zadání polohy bodu q slouží metoda `mousePressEvent`, kde se na základě stavu proměnné `self.__add_vertex` rozhoduje, zda kliknutí myší tvoří nový vrchol do aktuálního polygonu či přepisuje souřadnice hledaného bodu.

Následně je po stisknutí tlačítka *Analyze* spuštěna metoda `analyzePointAndPositionClick`, která si vyžádá uložený bod i seznam polygonů. S těmito daty pak pracují metody reprezentující oba řešené algoritmy: `getPointPolygonPositionRC` a `getPointPolygonPositionWN`

1. Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu

V první části úlohy je řešen *Point Location Problem* pomocí *Ray Crossing* algoritmu, který je realizován metodou `getPointPolygonPositionRC`. Ten postupně prochází všechny hrany zkoumaného polygonu a zároveň transformuje souřadnicový systém tak, aby bod q ležel v jeho počátku. Následně algoritmus pomocí podmínek kontroluje souřadnice y vrcholů a detekuje, zda daná úsečka protíná horizontální paprsek. Pokud ano, pak vypočítá polohu průsečíku a v případě, že tento průsečík leží kladným směrem od analyzovaného bodu, navýší počítadlo. Výpočet je po projití všech hran zakončen operací modulo, kdy lichý počet nalezených průsečíků potvrzuje, že bod leží uvnitř aktuálně testovaného polygonu.

```
#Process all polygon edges
for i in range(n):

    p_i = pol[i]
    p_i1 = pol[(i+1)%n]

    if self.PointOnBoundary(q, p_i, p_i1):
        return -1

    #Start point of the edge
    xi = pol[i].x() - q.x()
    yi = pol[i].y() - q.y()

    #End point of the edge
    xi1 = pol[(i+1)%n].x() - q.x()
    yi1 = pol[(i+1)%n].y() - q.y()

    #Find suitable segment
    if (yi1 > 0) and (yi<= 0) or (yi > 0) and (yi1 <= 0):

        #Compute intersection
        xm = (xi1 * yi - xi * yi1) / (yi1 - yi)

        #Correct intersection
        if xm > 0:

            #Increment number of intersections
            k = k + 1
```

Kód 1: Procházení hran polygonu pomocí for cyklu

Následuje grafické zvýraznění nalezeného polygonu ve frameworku QT. Ve chvíli, kdy algoritmus vrátí hodnotu reprezentující shodu, tak se spustí metoda `setHighlightedPolygon`. Pomocí překreslující události `paintEvent` se tento polygon vybarví tyrkysovou barvou.

2. Analýza polohy bodu (uvnitř, vně) metodou Winding Number Algorithm

Druhá část úlohy řeší *Point Location Problem* pomocí algoritmu *Winding Number*, a to prostřednictvím metody `getPointPolygonPositionWN`. Na počátku je inicializována proměnná `omega_sum = 0`, do které se budou postupně sčítat dílčí úhly, rovněž je definována toleranční hodnota `epsilon`. Následuje cyklus procházející všechny hrany polygonu, kde je do proměnných p_i a p_i1 vždy uložen počáteční a koncový vrchol aktuální úsečky. Pokud volání metody `PointOnBoundary` neodhalí, že bod q leží přesně na hranici (což by rovnou ukončilo výpočet s hodnotou -1), program přistoupí k výpočtu úhlu. Vytvoří dva vektory směřující od analyzovaného bodu k oběma vrcholům a pomocí euklidovských délek a skalárního součinu `dot_product` vypočítá hodnotu cosinus.

Ta je nejprve ořezána pomocí funkcí `max` a `min` do bezpečného intervalu, následně je z ní pomocí funkce `math.acos` získána velikost dílčího úhlu `omega_i`. Posléze je nutné určit prostorovou orientaci úhlu. K tomu slouží výpočet determinantu z vektorů definujících aktuální hranu a analyzovaný bod. Je-li hodnota determinantu kladná (bod leží vlevo), úhel se do celkové sumy přičte. Je-li záporná (bod leží vpravo), úhel se odečte.

3. Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu

V rámci implementace algoritmu *Winding Number* je stav, kdy bod leží na hraně polygonu řeše na počátku každého iteračního kroku v metodě `getPointPolygonPositionWN`. Ještě před zahájením výpočtu směrových vektorů a úhlů je volána evaluační metoda PointOnBoundary, jíž jsou jako argumenty předány souřadnice zkoumaného bodu q a obou definujících vrcholů aktuální orientované hrany p_i a p_i1. Uvnitř metody `PointOnBoundary` je detekce incidence bodu a hrany algoritmizována do dvou na sebe navazujících analytických fází: testu kolinearity a testu prostorového omezení.

```
def PointOnBoundary(self, q:QPointF, p1:QPointF, p2:QPointF):
epsilon = 0.0001

#Vertex test
if(abs(q.x() - p1.x()) < epsilon and abs(q.y() - p1.y()) < epsilon):
    return True

#Edge test
det = (p2.x() - p1.x()) * (q.y() - p1.y()) - (p2.y() - p1.y()) * (q.
   x() - p1.x())
if abs(det) < epsilon:
    min_x, max_x = min(p1.x(), p2.x()), max(p1.x(), p2.x())
    min_y, max_y = min(p1.y(), p2.y()), max(p1.y(), p2.y())
    if min_x - epsilon <= q.x() <= max_x + epsilon and min_y -
       epsilon <= q.y() <= max_y + epsilon:
        return True
return False
```

Kód 2: Řešení singulárního případu: bod na hraně polygonu

Následně je pomocí funkcí `min` a `max` vytvořena obálka hrany, která geometricky ověří, zda souřadnice bodu do této konkrétní úsečky spadají. Pokud jsou obě podmínky splněny, výpočet je pro daný polygon ukončen s návratovou hodnotou -1, díky čemuž uživatelské rozhraní dostane signál k odlišnému vizuálnímu zvýraznění topologické singularity.

4. Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu

Řešení je v případě *Ray Crossing* algoritmu identické, jako u algoritmu *Winding Number*. Na počátku cyklu je opět volána metoda `PointOnBoundary`.

5. Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů

Řešení topologické singularity, kdy je analyzovaný bod totožný s některým z vrcholů polygonu, je řešeno v rámci metody `PointOnBoundary`, která je volána hned v úvodu iteračního cyklu obou hlavních algoritmů, tedy u *Ray Crossing* i *Winding Number*. Samotný test shody s vrcholem představuje úplně první geometrické ověření uvnitř této metody. Výpočet zkoumá absolutní rozdíl souřadnic X a Y analyzovaného bodu q a počátečního vrcholu aktuální hrany $p1$. Tím je kolem každého uzlu definováno určité toleranční okolí. Spadají-li souřadnice zkoumaného bodu do tohoto vymezení, metoda vrací pravdivostní hodnotu $True$. Oba algoritmy na tento výsledek pak reagují ukončením výpočtu pro daný polygon s návratovou hodnotou -1.

6. Zvýraznění všech polygonů pro oba výše uvedené singulární případy

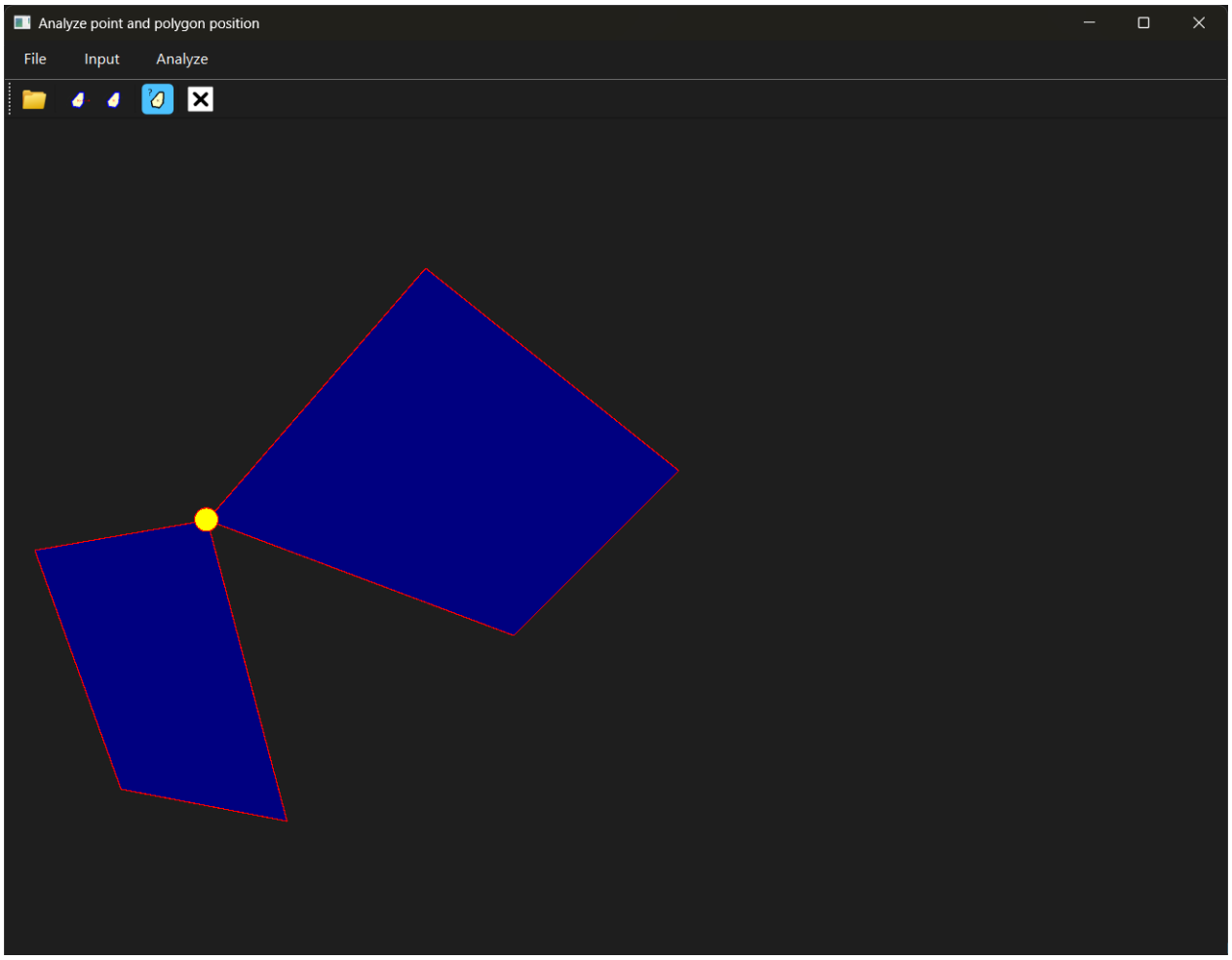
Jakmile evaluační metoda `PointOnBoundary` detekuje topologickou singularitu, oba hlavní algoritmy vygenerují specifickou návratovou hodnotu -1. Tato hodnota je následně zachycena v prohledávacím cyklu metody `analyzePointAndPositionClick` a rovněž i metody `analyzePointAndPositionWNClick`. Dotčený polygon je následně přidán do seznamu `highlighted_pols`.

Po otestování celé mapy program volá metodu `setHighlightedPolygons`. Společně se seznamem všech nalezených tvarů je do této metody předán i logický parametr s hodnotou True, který indikuje právě hraniční stav. Uvnitř vykreslovací metody `paintEvent` pak program vyhodnotí aktivní stav proměnné `self.__highlight_boundary` a namísto tyrkysové výplně změní vlastnosti grafického pera příkazem `qp.setPen(Qt.GlobalColor.red)`. Všechny dotčené polygony uložené v seznamu jsou následně pomocí cyklu vykresleny na plátno s červeným obrysem.

Závěr

Vytvořená aplikace představuje plně funkční geoinformatický nástroj pro analýzu prostorových vztahů mezi bodem a polygonovými daty v netopologickém vektorovém modelu. Grafické rozhraní umožňuje uživateli interaktivní digitalizaci prvků a následné spuštění výpočetních metod Ray Crossing nebo Winding Number, přičemž dosažený matematický výsledek je posléze interpretován formou kartografické vizualizace na plátně.

Aplikace však postrádá implementaci vstupně-výstupních operací. Přestože tak grafické uživatelské rozhraní obsahuje tlačítko pro otevření souboru, tato akce není v kódu propojena s žádnou funkcí a aplikace neumožňuje automatizované načítání bodů či polygonů z externích souborů.



Obrázek 4: Vzhled aplikace

Zdroje

O'ROURKE, J. (1998): Computational geometry in C. Cambridge University Press, Cambridge.

PREPARATA, F. P., SHAMOS, M. I. (1985): Computational geometry: An Introduction. Springer, New York.